

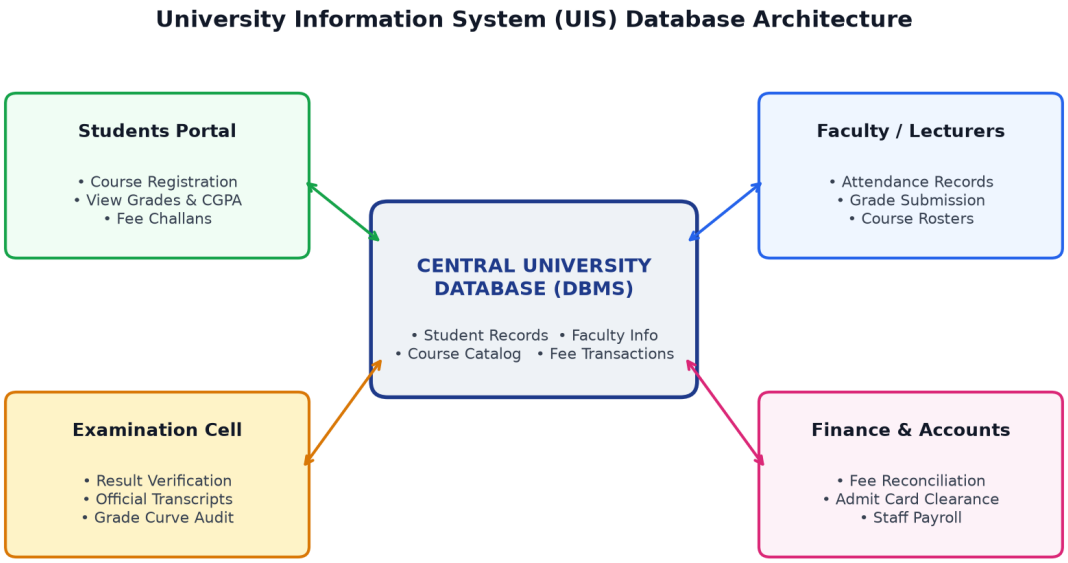
Name	Bheesham Kumar Sajnani
Roll Number	25F-DS-020
Section	25F-DS
Lab Title	Database Systems — Assignment 01
Subject	Database Systems
Instructor	Engr. Seema Bughio

Task 01: Database Role in Information Systems

Requirements / Details: Explain how a database supports a University Information System. Identify at least four types of information that can be stored in the database and explain how different users can use this information.

Solution:

A University Information System (UIS) relies on a centralized Relational Database Management System (DBMS) to operate reliably. Instead of keeping separate files or Excel sheets across departments which cause data mismatch and duplication, the database acts as a single central source of truth. It enforces referential integrity constraints so invalid data cannot be entered, allows hundreds of students and faculty to access the system at the same time without data loss, and restricts confidential data (like exam papers and salaries) using user permissions.



Types of Information Stored in University Database:

#	Information Type	Attributes Stored
1	**Student Records**	Student ID, Name, Department, Enrollment Year, Courses Enrolled, Attendance, CGPA.
2	**Faculty Records**	Faculty ID, Name, Designation, Department, Assigned Courses, Timetable, Salary Info.
3	**Course Catalog**	Course Code (e.g. DS-211), Title, Credit Hours, Prerequisites, Syllabus, Allocated Lab.

4	**Fee Transactions**	Voucher Number, Student ID, Due Date, Tuition Amount, Payment Status (Paid/Pending), Bank ID.

How Different Users Use This Information:

- **Students:** Check timetable, register for semester courses, view attendance, check exam grades and CGPA, and download fee vouchers.
- **Faculty / Lecturers:** Access enrolled class rosters, enter student lecture attendance, upload quiz/assignment/exam marks, and compute internal assessments.
- **Examination Cell:** Verify student grades, calculate grade distribution curves, verify prerequisite clearance, and print official transcripts.
- **Accounts Department:** Verify fee payments from banks, generate student clearance for admit cards, and process staff payroll.

Task 02: Basic Database Security

Requirements / Details: A university database contains students' personal information and examination results. Identify three basic security measures that should be implemented to protect this data. Explain the purpose of each measure.

Solution:

1. Authentication and Role-Based Access Control (RBAC): Users must log in with secure credentials (passwords with salted hashing and MFA for staff). The DBMS enforces least privilege: students only get SELECT permission on their own grades via views; teachers get UPDATE/INSERT rights only on courses they teach; admin/exam staff have audited access.

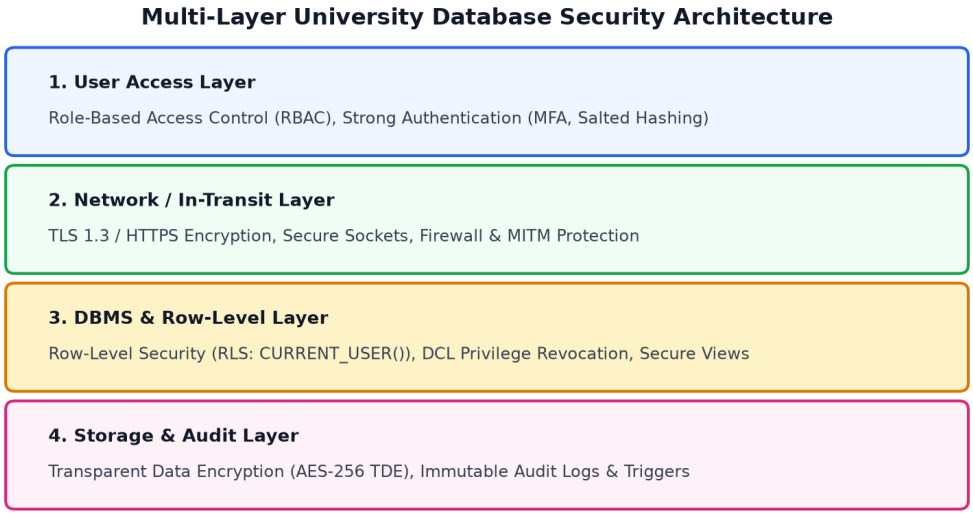
Purpose: Prevents unauthorized users or students from accessing sensitive records, modifying marks, or viewing peers' private personal data.

2. Data Encryption (In-Transit and At-Rest): Network traffic between web portals and the database is secured using TLS 1.3 / HTTPS. At the storage layer, Transparent Data Encryption (TDE with AES-256) encrypts database files and backups.

Purpose: Protects data confidentiality. Even if network packets are sniffed or a physical hard drive/backup is stolen, the data cannot be read without cryptographic keys.

3. Database Auditing, Activity Logging & Check Constraints: An automated audit log records every grade update (User_ID, old marks, new marks, timestamp, and IP). Check constraints (e.g. 0 <= Marks <= 100) ensure only valid values are accepted.

Purpose: Guarantees accountability, non-repudiation, and traceability. If an unauthorized grade alteration is attempted, administrators can see exactly who did it and rollback the changes.



Task 03: DBMS Operations (SQL Queries)

Requirements / Details: Given a Student table containing Student_ID, Name, Department, and CGPA, write SQL queries to perform the following operations: (1) Display students with CGPA > 3.0, (2) Change department of a particular student, (3) Remove a student from the table, (4) Add a new student.

Table Schema & Initial Records Setup:

QUERY:

```sql
-- Step 1: Create Student Table
CREATE TABLE Student (
    Student_ID VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Department VARCHAR(50) NOT NULL,
    CGPA DECIMAL(3, 2) NOT NULL CHECK (CGPA >= 0.00 AND CGPA <= 4.00)
);

-- Step 2: Populate Initial Sample Records
INSERT INTO Student (Student_ID, Name, Department, CGPA) VALUES
('DS-25-001', 'Sarah Ahmed', 'Data Science', 3.85),
('DS-25-014', 'Bilawal Raza', 'Data Science', 3.42),
('CS-25-022', 'Zohaib Hassan', 'Computer Science', 2.85),
('CS-25-045', 'Hamza Tariq', 'Computer Science', 3.10),
('SE-25-030', 'Fatima Noor', 'Software Engineering', 3.65),
('DS-25-099', 'Ali Murad', 'Data Science', 2.70);
```

OUTPUT:

```
Commands completed successfully.

Student_ID  Name           Department            CGPA
----------  -------------  --------------------  ----
DS-25-001   Sarah Ahmed    Data Science          3.85
DS-25-014   Bilawal Raza   Data Science          3.42
CS-25-022   Zohaib Hassan  Computer Science      2.85
CS-25-045   Hamza Tariq    Computer Science      3.10
SE-25-030   Fatima Noor    Software Engineering  3.65
DS-25-099   Ali Murad      Data Science          2.70

(6 rows affected)
```

Operation 01: Display students with CGPA greater than 3.0

Purpose: SELECT statement with WHERE clause filtering students where CGPA strictly exceeds 3.0.

QUERY:

```sql
-- Operation 1: Display students with CGPA greater than 3.0
SELECT Student_ID, Name, Department, CGPA
FROM Student
WHERE CGPA > 3.0;
```

OUTPUT:

```
Student_ID  Name           Department            CGPA
----------  -------------  --------------------  ----
DS-25-001   Sarah Ahmed    Data Science          3.85
DS-25-014   Bilawal Raza   Data Science          3.42
CS-25-045   Hamza Tariq    Computer Science      3.10
SE-25-030   Fatima Noor    Software Engineering  3.65

(4 rows affected)
```

Operation 02: Change the department of a particular student

Purpose: UPDATE statement modifying the Department column for student 'CS-25-045' uniquely by primary key.

QUERY:

```sql
-- Operation 2: Change department of a particular student
UPDATE Student
SET Department = 'Data Science'
WHERE Student_ID = 'CS-25-045';

-- Verification
SELECT Student_ID, Name, Department, CGPA
FROM Student
WHERE Student_ID = 'CS-25-045';
```

OUTPUT:

```
(1 row affected)

Student_ID  Name         Department    CGPA
----------  -----------  ------------  ----
CS-25-045   Hamza Tariq  Data Science  3.10

(1 row affected)
```

Operation 03: Remove a student from the table

Purpose: DELETE FROM statement removing student 'DS-25-099' uniquely by primary key.

QUERY:

```sql
-- Operation 3: Remove a student from the table
DELETE FROM Student
WHERE Student_ID = 'DS-25-099';

-- Verification
SELECT * FROM Student
WHERE Student_ID = 'DS-25-099';
```

OUTPUT:

```
(1 row affected)

Student_ID  Name  Department  CGPA
----------  ----  ----------  ----

(0 rows affected)
```

Operation 04: Add a new student

Purpose: INSERT INTO statement adding a new student record 'DS-25-102' (Ayesha Khan) into the Student table.

QUERY:

```
-- Operation 4: Add a new student
INSERT INTO Student (Student_ID, Name, Department, CGPA)
VALUES ('DS-25-102', 'Ayesha Khan', 'Data Science', 3.78);

-- Verification
SELECT Student_ID, Name, Department, CGPA
FROM Student
WHERE Student_ID = 'DS-25-102';
```

OUTPUT:

```
(1 row affected)

Student_ID  Name         Department    CGPA
----------  -----------  ------------  ----
DS-25-102   Ayesha Khan  Data Science  3.78

(1 row affected)
```

Task 04: Security Scenario

Requirements / Details: A student is able to access and modify another student's examination marks through the university database system. (a) What security problem exists? (b) What database security mechanism could help prevent this problem?

Solution:

Part (a): Security Problem Identified

1. Broken Access Control & Insecure Direct Object Reference (IDOR): The system fails to check if the user requesting the record owns or has permission for it. A student is able to target another student's record and modify it.
2. Violation of Principle of Least Privilege: Regular student user accounts have UPDATE and INSERT write permissions on examination marks tables, which should only belong to faculty and the exam cell.
3. Breach of Confidentiality and Integrity: Confidentiality is violated because private marks are visible to peers; Integrity is destroyed because unauthorized users can falsify academic grades.

Part (b): Database Security Mechanism to Prevent This

1. DCL Privilege Revocation: Revoke all UPDATE, INSERT, and DELETE permissions on Examination_Marks from the Student_Role.
2. Row-Level Security (RLS): Enable RLS policies in the DBMS so students can strictly SELECT rows where Student_ID = CURRENT_USER(). Any query attempting to access or modify another student's row is blocked at the database kernel level.
3. Secure Views and Stored Procedures: Force students to query a secure view (View_Student_Own_Marks) instead of the base table, and require teachers to submit grades via validated stored procedures.
4. Audit Triggers: Create an AFTER UPDATE trigger on examination marks that automatically logs any changes to an immutable audit table with timestamps.

QUERY (Security Implementation):

```
-- Security Defense Implementation
-- 1. Revoke dangerous DML operations from student role
REVOKE INSERT, UPDATE, DELETE ON Examination_Marks FROM Student_Role;

-- 2. Enforce Row-Level Security (RLS)
ALTER TABLE Examination_Marks ENABLE ROW LEVEL SECURITY;

CREATE POLICY student_view_own_marks ON Examination_Marks
    FOR SELECT TO Student_Role
    USING (Student_ID = CURRENT_USER());

-- 3. Grant marks modification strictly to assigned faculty
GRANT UPDATE (Marks, Grade) ON Examination_Marks TO Faculty_Role;
```

OUTPUT:

```
Commands completed successfully.
Security policy 'student_view_own_marks' applied.
Row Level Security enabled on 'Examination_Marks'.
Student_Role privileges restricted to SELECT on own records only.

(0 errors, 0 warnings)
```